Unit 4

Question:
Explain the fundamentals of exception handling in C++ using `try`, `catch`, and `throw`, with an example.

Answer:

- C++ exception handling is based on three keywords: `try`, `catch`, and `throw`.
- The code to be monitored for exceptions is placed inside a `try` block. If an exception occurs, it is thrown using `throw`, and the corresponding `catch` block handles it.

<u>Syntax:</u>

```
try {
  // try block
}
catch (type1 arg) {
  // catch block for type1
}
catch (type2 arg) {
  // catch block for type2
}
...
catch (typeN arg) {
  // catch block for typeN
}
```

- The `throw` statement is used to generate an exception:
  Syntax:

```
throw exception;
```

- When an exception is thrown, only the matching `catch` block (by data type) is executed; all others are bypassed.
- If no matching `catch` is found, the standard function `terminate()` is called, which by default invokes `abort()` and causes abnormal program termination.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

int main() {
  cout << "Start\n";
  try {
    cout << "Inside try block\n";
    throw 100;
    cout << "This will not execute";
  }
```

```
  catch (int i) {
    cout << "Caught an exception -- value is: ";
    cout << i << "\n";
  }
  cout << "End";
  return 0;
}
OUTPUT:
Start
Inside try block
Caught an exception -- value is: 100
End
```

- In this example, the integer `100` is thrown as an exception. Control is immediately transferred to the matching `catch(int i)` block, skipping any remaining statements in the `try` block.
- A `catch` block typically attempts to handle the error. If successful, program execution continues after the `catch`. If not, the program may terminate using `exit()` or `abort()`.
- If no exception is thrown, the `catch` block is bypassed entirely.

Question:
Can an exception be thrown from outside the try block in C++? Explain with an example.

Answer:

- An exception can be thrown from outside the try block, provided it is thrown by a function that is called from within the try block.
- In such cases, control transfers from the function to the matching `catch` block defined after the `try`.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

void Xtest(int test) {
  cout << "Inside Xtest, test is: " << test << "\n";
  if(test) throw test;
}

int main() {
  cout << "Start\n";
  try {
    cout << "Inside try block\n";
    Xtest(0);
```

```
    Xtest(1);
    Xtest(2);
  }
  catch (int i) {
    cout << "Caught an exception -- value is: ";
    cout << i << "\n";
  }
  cout << "End";
  return 0;
}
OUTPUT:

Start
Inside try block
Inside Xtest, test is: 0
Inside Xtest, test is: 1
Caught an exception -- value is: 1
End
```

Question:
What happens when a try block is localized to a function in C++? Illustrate with an example.

Answer:

- A try block can be localized to a function.
- This means that the exception handling defined inside the function applies only during that function call and is reset every time the function is entered.
- Each call to the function executes its own try-catch logic independently of previous or subsequent calls.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

// Localize a try/catch to a function.
void Xhandler(int test) {
  try {
    if(test) throw test;
  }
  catch(int i) {
    cout << "Caught Exception #: " << i << '\n';
  }
}

int main() {
  cout << "Start\n";
```

```
  Xhandler(1);
  Xhandler(2);
  Xhandler(0);
  Xhandler(3);
  cout << "End";
  return 0;
}
OUTPUT:

Start
Caught Exception #: 1
Caught Exception #: 2
Caught Exception #: 3
End
```

Question:
How can user-defined class types be used in C++ exception handling? Explain with an example.

Answer:

- Exceptions in C++ can be of any type, including user-defined class types.
- Most exceptions are class types to encapsulate detailed information about the error.
- Creating an exception class allows the exception object to carry descriptive data, which can then be used by the `catch` block to process the error effectively.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
#include <cstring>
using namespace std;

class MyException {
public:
  char str_what[80];
  int what;

  MyException() { *str_what = 0; what = 0; }

  MyException(char *s, int e) {
    strcpy(str_what, s);
    what = e;
  }
};

int main() {
  int i;
  try {
```

```
    cout << "Enter a positive number: ";
    cin >> i;
    if(i < 0)
      throw MyException("Not Positive", i);
  }
  catch (MyException e) {
    cout << e.str_what << ": ";
    cout << e.what << "\n";
  }
  return 0;
}
OUTPUT:
Enter a positive number: -4
Not Positive: -4
```

- In this program, if the user enters a negative number, an object of the `MyException` class is thrown.
  This object contains both a message and the value that caused the exception.
- The `catch` block receives the object and prints the descriptive error.

Question:
How can multiple `catch` statements be used in C++ exception handling? Explain with an example.

Answer:

- A single `try` block in C++ can be followed by multiple `catch` statements.
- Each `catch` must handle a different type of exception.
- `catch` blocks are checked in the order they appear.
- Only the first matching catch block is executed; all others are ignored.
- This allows handling various error types separately and appropriately.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;
// Different types of exceptions can be caught.

void Xhandler(int test) {
  try {
    if(test)
      throw test;           // throw int
    else
      throw "Value is zero"; // throw string literal
  }

  catch(int i) {
```

```
    cout << "Caught Exception #: " << i << '\n';
  }
  catch(const char *str) {
    cout << "Caught a string: ";
    cout << str << '\n';
  }
}

int main() {
  cout << "Start\n";
  Xhandler(1);
  Xhandler(2);
  Xhandler(0);
  Xhandler(3);
  cout << "End";
  return 0;
}
OUTPUT:
Start
Caught Exception #: 1
Caught Exception #: 2
Caught a string: Value is zero
Caught Exception #: 3
End
```

- In this example:
  - If `test` is non-zero, an integer is thrown and caught by `catch(int i)`.
  - If `test` is zero, a string literal is thrown and caught by `catch(const char* str)`.
- The use of multiple `catch` statements allows precise handling of different exception types.

Question:
How are derived-class exceptions handled in C++? What is the correct order of `catch` blocks when using inheritance?

Answer:

- In C++, a `catch` block for a base class will also match exceptions of any class derived from it.
- Therefore, to handle derived-class exceptions separately, the `catch` block for the derived class must come before the one for the base class.
- If the base-class `catch` is placed first, it will catch both base and derived exceptions, preventing the derived-class handler from executing.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
```

```
using namespace std;

class B {
};

class D : public B {
};

int main() {
  D derived;
  try {
    throw derived;
  }
  catch(D d) {
    cout << "Caught a derived class.\n";
  }
  catch(B b) {
    cout << "Caught a base class.\n";
  }
  return 0;
}
OUTPUT:
Caught a derived class.
```

- In this corrected code:
  - The object `derived` of class `D` is thrown.
  - `catch(D d)` matches the exception and executes.
  - `catch(B b)` is bypassed.
- This order ensures that the most specific match (derived class) is checked before the more general one (base class), allowing precise exception handling.

Question:
How can C++ catch all exceptions regardless of their type? Explain with code and its output.

Answer:

In C++, you can catch all exceptions, regardless of their type, using a generic catch block:
Syntax:

```
catch(...) {
  // process all exceptions
}
```

- The ellipsis `...` matches any thrown exception type—whether built-in or user-defined.
- This form is especially useful:
  - When the specific exception types are unknown.

- As the last handler in a group of `catch` blocks to act as a default.
  can write this in last and write an example code for this. On page 12 of c++ unit 4.pdf
- To prevent abnormal termination from unhandled exceptions.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

void Xhandler(int test) {
  try {
    if(test == 0) throw test;        // throw int
    if(test == 1) throw 'a';         // throw char
    if(test == 2) throw 123.23;      // throw double
  }
  catch(...) {                       // catch all exception
types
    cout << "Caught One!\n";
  }
}

int main() {
  cout << "Start\n";
  Xhandler(0);
  Xhandler(1);
  Xhandler(2);
  cout << "End";
  return 0;
}
OUTPUT:

Start
Caught One!
Caught One!
Caught One!
End
```

Question:
How can C++ restrict the types of exceptions a function is allowed to throw? Explain with code and output.

Answer:

- In C++, you can restrict the types of exceptions a function is allowed to throw outside itself using a `throw` specification in the function declaration.

<u>Syntax:</u>

```
ret-type func-name(arg-list) throw(type1, type2, ..., typeN)
{
  // function body
}
```

- Only the exception types listed in the `throw(type-list)` clause may be thrown out of the function.
- If the function throws an unsupported type, the standard function `unexpected()` is called, which by default invokes `abort()` and terminates the program.
- If you want to prevent a function from throwing any exceptions, specify an empty `throw()` list. *can write example code*
- **Important Note:** The restriction applies only to exceptions propagated outside the function. A function may still throw any type within itself as long as the exception is caught locally.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

// This function can only throw int, char, and double
void Xhandler(int test) throw(int, char, double) {
  if(test == 0) throw test;      // throw int
  if(test == 1) throw 'a';       // throw char
  if(test == 2) throw 123.23;    // throw double
}

int main() {
  cout << "start\n";
  try {
    Xhandler(0);                 // can try with 1 or 2 as well
  }
  catch(int i) {
    cout << "Caught an integer\n";
  }
  catch(char c) {
    cout << "Caught char\n";
  }
  catch(double d) {
    cout << "Caught double\n";
  }
  cout << "end";
  return 0;
}
OUTPUT (for Xhandler(0)):
start
Caught an integer
end
```

Question:
How can an exception be rethrown in C++? Explain with code and output.

Answer:

- In C++, an exception can be rethrown from within a `catch` block using the `throw;` statement with no argument.
- Rethrowing allows an exception to be passed on to an outer `try-catch` block, enabling multiple handlers to process the same exception at different levels.
- A rethrow must occur:
  - Inside a `catch` block, or
  - From a function called within a `catch` block.
- The rethrown exception will not be caught again by the same `catch` block that originally handled it.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

void Xhandler() {
  try {
    throw "hello"; // throw a char *
  }
  catch(const char *) {
    cout << "Caught char * inside Xhandler\n";
    throw; // rethrow the exception
  }
}

int main() {
  cout << "Start\n";
  try {
    Xhandler();
  }
  catch(const char *) {
    cout << "Caught char * inside main\n";
  }
  cout << "End";
  return 0;
}
OUTPUT:
Start
Caught char * inside Xhandler
Caught char * inside main
End
```

Question:
What are the roles of `terminate()` and `unexpected()` in C++ exception handling? When are they invoked?

Answer:

- `terminate()` and `unexpected()` are functions provided by the Standard C++ Library for handling severe errors during exception processing.
- Their prototypes are:
```
void terminate();
void unexpected();
```
- These functions require the `<exception>` header.

<u>`terminate()` is invoked when:</u>

  - No matching `catch` block is found for a thrown exception.
  - A rethrow (`throw;`) is attempted when no exception is currently active.
  - During stack unwinding, if a destructor throws an exception while another exception is already in progress.
- In such cases, `terminate()` acts as the handler of last resort.
- By default, `terminate()` calls `abort()`, which terminates the program abnormally.

<u>`unexpected()` is invoked when:</u>

  - A function throws an exception that is not listed in its exception specification (`throw(type-list)`).
- By default, `unexpected()` calls `terminate()`, leading to abnormal termination.

Question:
How can the default `terminate()` and `unexpected()` handlers be replaced in C++? Explain with an example and its output.

Answer:

- In C++, you can override the default behavior of `terminate()` and `unexpected()` by assigning custom handler functions using `set_terminate()` and `set_unexpected()`.
- Both functions require the `<exception>` header.

<u>To change the `terminate()` handler:</u>

```
terminate_handler set_terminate(terminate_handler newhandler)
throw();
```

- `newhandler` is a pointer to the user-defined function that should replace the default handler.

- The custom terminate handler must stop program execution. It must not return or resume execution.

<u>To change the `unexpected()` handler:</u>

```
unexpected_handler set_unexpected(unexpected_handler
newhandler) throw();
```

- Similar rules apply. It may stop execution, call `terminate()`, or throw an allowed exception.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
#include <cstdlib>
#include <exception>
using namespace std;

void my_Thandler() {
  cout << "Inside new terminate handler\n";
  abort(); // stop program execution
}

int main() {
  // Set a new terminate handler
  set_terminate(my_Thandler);

  try {
    cout << "Inside try block\n";
    throw 100; // throws int
  }
  catch (double i) { // does not match int
    // this catch block will not execute
  }

  return 0;
}
OUTPUT:
Inside try block
Inside new terminate handler
abnormal program termination
```

Question:
What is the purpose of the `uncaught_exception()` function in C++ exception handling?

Answer:

- The function `uncaught_exception()` is provided by the C++ exception handling subsystem.

<u>Syntax:</u>

```
bool uncaught_exception();
```

- It returns `true` if an exception has been thrown and is currently in the process of being handled (i.e., not yet caught).
- Once the exception is caught, `uncaught_exception()` returns `false`.
- This function can be useful for checking whether an exception is active, especially inside destructors or cleanup code, to decide if specific actions should be taken during stack unwinding.

Question:
What are the `exception` and `bad_exception` classes in C++? When is `bad_exception` used?

Answer:

- In C++, the Standard Library provides a base class named `exception` for all exceptions it defines.
- When a standard library function throws an exception, it throws an object derived from the `exception` base class.
- These classes are defined in the `<exception>` header.
- One such derived class is `bad_exception`.
- **`bad_exception` is specifically used when:**
    - A function has an exception specification (i.e., it uses `throw(type-list)`).
    - If that function throws an exception not listed in the specification, the `unexpected()` function is called.
    - If the `unexpected()` handler is set to throw a `bad_exception`, it will be thrown as a fallback.
- This allows a program to recover or report errors gracefully when a function violates its declared throw list.
- Both `exception` and `bad_exception` are used to standardize exception handling and provide meaningful error types within the C++ Standard Library.

Question:
Differentiate between an error and an exception in C++ with respect to definition, severity, handling, and recoverability. Include examples and a comparison table.

Answer:

<u>Definition:</u>

- Error: A serious issue that occurs during compilation or runtime due to system limitations or bugs. Errors are usually irrecoverable.
- Exception: A problem during program execution that can be anticipated and handled using exception-handling mechanisms in C++.

Severity:

- Error: Often fatal. Causes abnormal termination.
  Examples: segmentation fault, stack overflow.
- Exception: Less severe. Can be handled gracefully.
  Examples: divide-by-zero, file not found.

Handling Mechanism:

- Error: Not handled using exception handling. Usually handled at the system or compiler level.
- Exception: Handled using `try`, `catch`, and `throw`. Allows structured error management.

Programming Philosophy:

- Error: Not meant to be handled using exceptions. Routine issues like end-of-file or null strings should be handled through logic, not exceptions.
- Exception: Designed for unusual or unexpected runtime events. Supports a termination model and stack unwinding.

Example:

Error:

```
int x = 10 / 0; // Runtime error — program crashes
```

Exception:

```
try {
    if (den == 0) throw "Division by zero!";
} catch (const char *msg) {
    cout << msg;
}
```

Recoverability:

- Error: Not recoverable. Program typically halts.
- Exception: Recoverable using exception handling. Program can continue.

Comparison Table:

Aspect	Error	Exception

Meaning	Serious issue or fault	Runtime irregular condition
Occurs At	Compile time or runtime	Always at runtime
Recoverable?	Usually no	Yes, using `try-catch`
Handled By	OS / compiler / crash	`try`, `catch`, `throw`
Example	Stack overflow, syntax error	Divide-by-zero, out-of-bounds access
Fault Tolerance	Low	High

Question:
Write a C++ program to handle divide-by-zero using exception handling along with the output.

Answer:

```
INPUT CODE:
#include <iostream>
using namespace std;

void divide(double a, double b);

int main()
{
    double i, j;
    do {
        cout << "Enter numerator (0 to stop): ";
        cin >> i;
        cout << "Enter denominator: ";
        cin >> j;
        divide(i, j);
    } while(i != 0);

    return 0;
}
```

```
void divide(double a, double b)
{
    try {
        if(!b) throw b;            // check for divide-by-zero
        cout << "Result: " << a/b << endl;
    }
    catch (double b) {
        cout << "Can't divide by zero.\n";
    }
}
OUTPUT:
Enter numerator (0 to stop): **10**
Enter denominator: **2**
Result: 5

Enter numerator (0 to stop): **5**
Enter denominator: **0**
Can't divide by zero.

Enter numerator (0 to stop): **0**
Enter denominator: **4**
Result: 0
```

Question:
List and explain the different types of exceptions in C++ with suitable examples.

Answer:

1. Built-in Type Exceptions

- C++ allows exceptions of any primitive type.
- Common types: `int`, `char`, `double`, `const char*`.
- These are used in basic programs to demonstrate the mechanics of exception handling.
- Example:

```
throw 100;          // int
throw 'a';          // char
throw 123.23;       // double
throw "error";      // string literal
```

2. Class Type Exceptions (User-defined)

- Objects of user-defined classes can be thrown as exceptions.
- Used to encapsulate detailed error information.
- Enables object-oriented exception handling.
- <u>Example:</u>

```
class MyException {
public:
    char str_what[80];
    int what;
    MyException(char *s, int e) {
        strcpy(str_what, s);
        what = e;
    }
};
throw MyException("Not Positive", i);
```

3. Standard Exception Classes (System-defined)

- These are defined in `<exception>` and derived from the base class `exception`.
- Common types:
    - `bad_alloc`: thrown by `new` on memory allocation failure.
    - `bad_cast`: thrown by invalid `dynamic_cast`.
    - `bad_typeid`: thrown by `typeid` on null pointer to polymorphic type.
    - `bad_exception`: thrown by `unexpected()` if declared in a throw list.
- When a function supplied by the C++ standard library throws an exception, it will be an object derived from the base class `exception`.

4. Ellipsis-Based Catch-All Exception (`catch(...)`)

- Used to catch exceptions of any type.
- Acts as a default handler if no other `catch` matches.
- Typically placed last in a series of catch blocks.
- <u>Example:</u>

```
catch(...) {
    // handles any exception
}
```

5. Derived-Class Exceptions

- If a class `D` is derived from class `B`, and both types may be thrown as exceptions, the order of `catch` blocks matters.
- `catch(D d)` must appear before `catch(B b)`. Otherwise, the base class catch will also handle derived class exceptions.
- A catch clause for a base class will also match any class derived from that base.

Question:
What is the system exception-based class in C++? Explain the purpose of the `exception` class, its key function, its derived types, and show an example of usage.

Answer:

1. Base Exception Class

- The standard base class for all system exceptions in C++ is `exception`.
- Declared in the `<exception>` header.
- It acts as the root for all standard exception types provided by the language.

2. Purpose and Functionality

- `exception` provides a virtual function:

```
virtual const char* what() const throw();
```

- This function returns a C-style string describing the exception.
- It allows all derived exceptions to provide a consistent interface for error reporting.

3. Common Derived Classes

Several predefined classes inherit from `exception`, including:

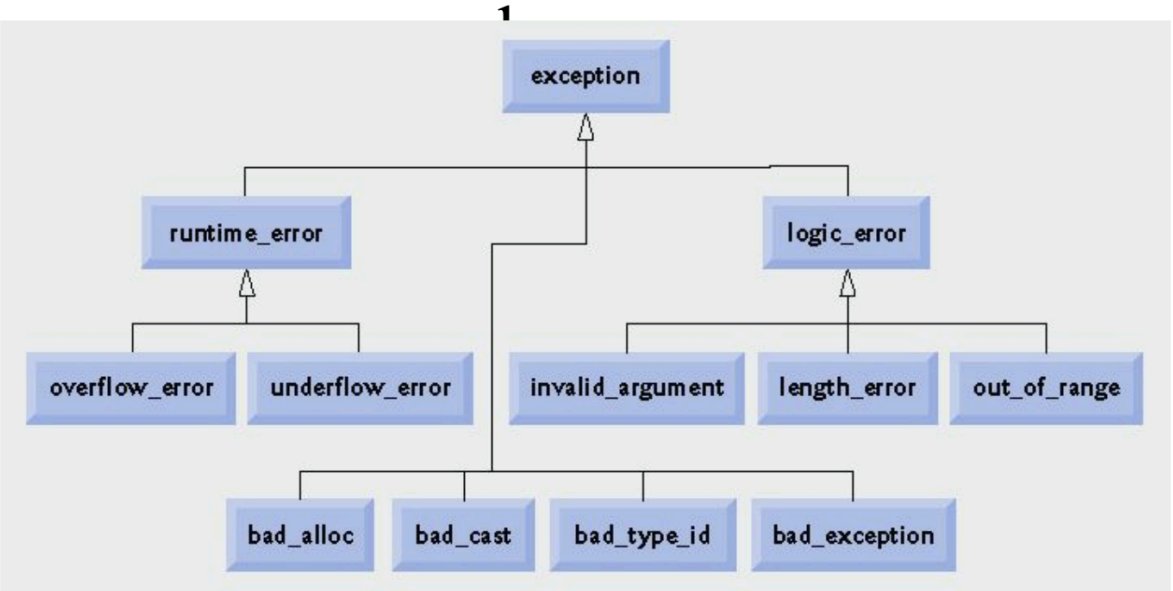
- `bad_alloc` – thrown when dynamic memory allocation using `new` fails.
- `bad_cast` – thrown when a `dynamic_cast` fails at runtime.
- `bad_typeid` – thrown when `typeid` is used on a null pointer to a polymorphic type.
- `bad_exception` – thrown when a function throws an exception not listed in its `throw()` specification.

4. Example: Handling `bad_alloc`

```
INPUT CODE:
 #include <iostream>
#include <new>
using namespace std;

int main() {
    try {
        int* p = new int[100000000000];  // attempt very large
allocation
    }
    catch (bad_alloc &e) {
        cout << "Exception caught: " << e.what() << endl;
    }
}
```

- This program tries to allocate an excessive amount of memory.
- If the allocation fails, `bad_alloc` is thrown and caught.
- The `what()` function provides the error message.

5. Importance of System Exception Classes

- They simplify error handling by providing built-in, descriptive exceptions.
- Help in writing cleaner and more robust programs without manually defining custom error messages.
- Enable consistent handling of various runtime problems using a unified interface.